Fixed-arity nondeterministic remainder programs: expanded proof and executable checks

Research note accompanying the uniform-process construction

October 16, 2026

Abstract

The Euclidean-prefix construction gives a family of finite nondeterministic recursive programs with arbitrarily small logarithmic remainder-call coefficients. Each program uses only remainder and the zero test, and its arity is fixed independently of the inputs. This does not give a single sublogarithmic program. We give the program schema, correctness and cost arguments, and an executable compilation into first-order syntax. Lean checks the fixed-register recurrence, the asymptotic program bound, and translations preserving remainder counts into finite first-order syntax and a specialized two-stack recursive machine. It also checks correctness of every terminating syntactic evaluation and existence of a successful evaluation on every input.

1 The result and its quantifiers

We use the nondeterministic convention of Moschovakis: every convergent computation must return the correct value, divergent computations are disregarded, and remainder complexity is the minimum number of remainder calls in a convergent computation [1, §2E, (2E-2), (2E-5)]. All logarithms below have base 2.

Theorem 1. *For each integer $p \geq 1$, there is one finite nondeterministic recursive program P_p over $(\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0)$ computing the gcd such that*

$$\limsup_{a\to\infty} \max_{1\leq b\leq a} \frac{c_{P_p}(a,b)}{\log a} \leq \frac{3}{p}.$$

Its generator has $W = 4^p + 5$ individual parameters; the largest auxiliary arity is $W + 1$. In particular,

$$\forall \varepsilon > 0\ \exists P, B\ \forall a \geq b \geq 1, \qquad c_P(a,b) \leq \varepsilon \log a + B.$$

The program may depend on ε, but not on a, b. This is compatible with the Main Conjecture, whose lower-bound coefficient may depend on the program [1, p. 2]. No passage from this family to one sublogarithmic program is asserted. The bound concerns remainder calls, not internal recursive-machine steps or the time needed to find a successful branch.

2 A finite program for each fixed width

Fix $W \geq 2$ and write $\vec{x} = (x_0, \ldots, x_{W-1})$. Define the following finite family of equations:

$$\begin{aligned}
\mathrm{choose}(\vec{x}) &= x_i && (0 \leq i < W),\\
\mathrm{set}_i(\vec{x}, z) &= \mathrm{gen}(x_0, \ldots, x_{i-1}, z, x_{i+1}, \ldots, x_{W-1}) && (0 \leq i < W),\\
\mathrm{gen}(\vec{x}) &= \mathrm{choose}(\vec{x}),\\
\mathrm{gen}(\vec{x}) &= \mathrm{set}_i(\vec{x}, \mathrm{rem}(\mathrm{choose}(\vec{x}), \mathrm{choose}(\vec{x}))) && (0 \leq i < W),\\
\mathrm{gen}(\vec{x}) &= \mathrm{set}_i(\vec{x}, \mathrm{gen}(\vec{x})) && (0 \leq i < W),\\
\mathrm{gen}(\vec{x}) &= \mathrm{gen}(\mathrm{choose}(\vec{x}), \ldots, \mathrm{choose}(\vec{x})).
\end{aligned}$$

There are exactly W occurrences of choose in the last equation. The repeated definitions express finite nondeterministic choice. Each index i is a syntactic index: after fixing W, all the displayed families are expanded into finitely many ordinary equations. The symbols W, i are not runtime data, and there is no array-access, list-construction, or numeric-constant primitive.

The last equation copies or permutes any available parameters without a remainder call. In the subcomputation equation $\mathrm{set}_i(\vec{x}, \mathrm{gen}(\vec{x}))$, call-by-value evaluates the inner generator once and retains the caller's parameters until its return. The result replaces one parameter, and the outer generator continues. The Lean presentation inlines the setters: for example, the remainder alternative becomes $\mathrm{gen}(\vec{x}) = \mathrm{gen}(x_0, \ldots, x_{i-1}, \mathrm{rem}(\mathrm{choose}(\vec{x}), \mathrm{choose}(\vec{x})), x_{i+1}, \ldots, x_{W-1})$. The subcomputation alternative is inlined in the same way. Its fixed signature has four internal symbols: choose, gen, accept, and loop. The executable Python presentation retains the setters. These are ordinary first-order calls; each recursive variable has fixed arity [1, §§2A, 2D]. The recursion stack may grow, so W is not a bound on the total number of values in the entire computation.

Complete the program by setting $\mathrm{loop}(z) = \mathrm{loop}(z)$ and

$$\mathrm{accept}(a, b, g) = \begin{cases} \mathrm{loop}(g), & \mathrm{eq}_0(g),\\ g, & \mathrm{eq}_0(\mathrm{rem}(a, g)) \text{ and } \mathrm{eq}_0(\mathrm{rem}(b, g)),\\ \mathrm{loop}(g), & \text{otherwise.} \end{cases}$$

The cases are nested conditionals: the zero test is performed before either division, and the conjunction is expanded into two conditionals. Finally set

$$P_W(a, b) = \begin{cases} a, & \mathrm{eq}_0(b),\\ \mathrm{accept}(a, b, \mathrm{gen}(a, b, a, \ldots, a)), & \text{otherwise.} \end{cases}$$

Only rem and eq_0 are primitive symbols in this program. On a successful computation with $b > 0$, verification adds exactly two remainder calls and the whole program makes four zero tests.

3 Correctness and the operational interpretation

Proposition 2. *For every $W \geq 2$, every convergent computation of $P_W(a, b)$ returns $\gcd(a, b)$, and at least one such computation exists on every natural input. Positive scaling preserves the generator's successful choice sequences and remainder-call counts.*

Proof. Induct on a finite generator evaluation. Every common divisor of its input parameters divides its output: this is immediate for a selected parameter, is preserved by remainder and copying, and passes through a recursive subcomputation and its continuation.

Consequently $\gcd(a, b)$ divides every candidate g generated at the root. A successful check requires $g > 0$, $g \mid a$, and $g \mid b$, so also $g \mid \gcd(a, b)$. Thus $g = \gcd(a, b)$. The separate zero-input branch is correct as well.

There is always a successful generator computation following Euclid. With the current pair in the first two parameters, overwrite the first by its remainder modulo the second, then swap and duplicate parameters to obtain the next pair. When the second is zero, return the first. The choice sequence is selected in the existence proof; no additional test is introduced into the generator. Finally $\mathrm{rem}(gx, gy) = g\,\mathrm{rem}(x, y)$ for $g > 0$, and nonzero divisors remain nonzero under scaling. Induction preserves the same computation and its cost. □

The Lean relation `ShortGCD.RegisterProgram.Run` has four constructors: return a parameter, overwrite one parameter by a remainder, route parameters, or run a subcomputation and overwrite one parameter by its result. These correspond to the four generator alternatives above. For a remainder step, the two selector calls choose its operands; for a routing step, the W selector calls choose its argument tuple. Internal selector calls make no remainder calls.

The kernel-checked theorem `Syntax.run_invokes` translates every register run into an evaluation of the inlined first-order generator, preserving its remainder count exactly. `Syntax.accepts_eval` includes the verification wrapper. Independently, `Syntax.eval_head_correct` proves correctness of *every* terminating evaluation, and `Syntax.eval_head_exists` proves existence of a successful evaluation on every input. Thus correctness is not restricted to the selected fast witnesses.

The syntax contains variables, remainder, zero-test conditionals, and calls to a fixed finite signature; each symbol has a finite set of bodies. `Syntax.Eval` uses call-by-value evaluation trees, evaluating all arguments before the chosen body and summing their remainder counts. `RegisterSmallStep` gives an explicit two-stack machine for this language: it evaluates arguments right to left and has separate transitions for zero tests and branching. Its closures encode substituted source terms, not a new data type available to the program. The Python compiler and interpreter are independently checked executable artifacts, not verified implementations of the Lean semantics.

For these pure terms the evaluation trees correspond, with the same result and remainder count, to the book's recursive machine. In the forward direction, induct on a tree and serialize its argument evaluations right to left before entering the chosen body. Transition locality embeds each evaluation in its caller context [1, Table 1 and Lemma 2D.1, pp. 75–76]. A remainder node adds one remainder call; a zero-test conditional adds none and evaluates only its chosen branch. Conversely, decompose a terminating machine computation into its completed argument computations and selected body or branch, and induct on its length. Each selector occurrence makes its own choice, with no shared mutable state. The setters in this note merely pass their evaluated arguments to the generator, so inlining them also preserves results and remainder counts. `Syntax.Machine.eval_iff_steps` now checks the tree-to-machine equivalence in both directions; `Machine.Step.context` checks transition locality. The machine specializes the book's transition table to the fixed language and signature, rather than formalizing arbitrary structures and vocabularies. The separate equivalence with the Python presentation retaining setters remains a mathematical argument.

4 The fixed-prefix cost bound

We use the Euclidean-prefix and three-call contraction lemmas from the companion manuscript [2]. Fix $p \geq 1$, and put

$$k = 2^p, \qquad h = k^2 + 1 = 4^p + 1, \qquad W = h + 4.$$

The entire prefix $r_0, \ldots, r_h$ fits in $h+1$ generator parameters. Three further parameters suffice for the current modulus and two temporary residues. Each contraction uses at most three remainder calls and can overwrite these temporary parameters, leaving the prefix available.

After reaching a small modulus m, form $u = a \bmod m$ and $v = b \bmod m$ in the two temporary positions. Use a recursive generator subcomputation to produce $d = \gcd(m, u)$, first routing its parameters to copies of m, u. Its caller retains v. On return, replace the modulus parameter by d, then route to copies of d, v for the second gcd computation. The identity

$$\gcd(\gcd(m, a \bmod m), b \bmod m) = \gcd(m, \gcd(a, b))$$

recovers the original gcd, since every generated modulus is divisible by it. Only one result crosses each subcomputation boundary. In the early-small-remainder case, route directly to the next Euclidean pair. Thus the two recursive branches of the certificate proof are realizable in one fixed generator of width W, with the same remainder counts.

For precision, let $T_W(n)$ be the maximum, over $a, b \le 2^n$, of the minimum generator remainder count needed to produce $\gcd(a, b)$ from the initial tuple $(a, b, a, \ldots, a)$. Euclid gives $T_W(n) = O(n)$, and copying gives symmetry. Positive scaling lets us analyze coprime inputs and transport the resulting computations to general inputs without extra calls.

Fix $0 < \beta < 1/2$ and set $\alpha = 1 - \beta/2$. Use the early threshold N^α and the target modulus N^β, with $N = 2^n$. If the prefix is large, then $2a/r_h < 2N^{1-\alpha} < N^\beta$ for sufficiently large n. The prefix therefore occupies at least $k+1$ residue classes modulo every modulus used before stopping. A contraction decreases the modulus by a factor greater than k, and $J = \lceil n/p \rceil$ contractions suffice. It follows that

$$T_W(n) \le \max\left\{T_W(\lceil \alpha n \rceil) + h + 1, \quad 2T_W(\lceil \beta n \rceil) + h + 3\lceil n/p \rceil + 2\right\}.$$

All thresholds and operand choices select witnesses among computations of the same fixed program; β is not a program parameter.

Write $L_W = \limsup_{n\to\infty} T_W(n)/n$, finite and nonnegative by Euclid. Here h is fixed, so dividing the recurrence by n and taking upper limits gives

$$L_W \le \max\{\alpha L_W,\ 2\beta L_W + 3/p\}.$$

Since $\alpha < 1$, we obtain $(1 - 2\beta)L_W \le 3/p$. Letting $\beta \downarrow 0$ yields $L_W \le 3/p$. Taking $n = \lceil \log a \rceil$ and adding the two verification calls proves the first assertion of Theorem 1. For the second, choose $p > 3/\varepsilon$ and absorb the finitely many smaller inputs in B.

This argument uses a fixed h, unlike the sublogarithmic certificate construction. The price for a smaller logarithmic coefficient is a larger finite program. Neither the stated coefficient nor the parameter count is claimed optimal.

5 What still prevents the direct sublogarithmic compilation

The original construction uses a prefix length growing with input size. Replacing W by a function of the input in the compiler would produce different programs on different inputs. It would not prove the desired single-program result.

Saving a growing prefix in nested calls also does not directly implement the reservoir. A current function body can use its individual parameters; values retained in older continuations are not an indexed store. Returning exposes an older continuation but passes only the return value from the completed call. For example, a recursive prefix builder that carries only the current Euclidean pair

leaves the innermost call without all earlier remainders as parameters. The reservoir lemma cannot be applied there as though those values were freely available.

Regenerating the whole prefix for every contraction is too expensive for this construction: a positive prefix $r_0, \ldots, r_h$ requires $h-1$ remainder calls, so J rebuilds cost $(h-1)J$ calls before the contractions. For h of order $\sqrt{n}$ and J of order $n/\log n$, this is of order $n^{3/2}/\log n$.

This diagnoses the direct implementations, not every possible recursive program. A different organization of the computations might avoid the need for a fully reusable growing prefix. No impossibility theorem for such an organization has been proved here. In particular, the quantifier exchange

$$\forall \varepsilon\ \exists P_\varepsilon \qquad \text{to} \qquad \exists P\ \forall \varepsilon$$

remains unjustified.

6 Executable checks and current status

`register_program.py` compiles a fixed-width instance to an AST containing only individual variables, function calls, and conditionals. A static checker checks arities and sorts and rejects extra primitives and numeric literals. A separate evaluator implements call-by-value, with right-to-left argument evaluation. It executes the compiled syntax and counts actual remainder calls. The offline planner supplies nondeterministic choices; its unrestricted arithmetic is not executed by the program.

The suite reuses each of the three compiled programs of widths $9, 21, 69$ across 138 cases, including zero, unordered, scaled, random, and Fibonacci inputs. It additionally replays 129 positive-input cases after scaling both inputs by 13 with exactly the same choices. The suite exercises 2,086 contractions and 101 recursive subcomputations, with nesting depth up to four. Eight negative checks cover rejected candidates, zero divisors, malformed choices, and invalid program syntax. A two-register instance separately replays Euclid.

On one seeded 2,048-bit pair, whose ordinary Euclidean computation takes 1,188 remainder calls, the fixed programs give:

p	Generator parameters	Remainder calls	Internal calls
1	9	1,036	4,476
2	21	713	3,564
3	69	634	4,920

The remainder counts include final verification. These are successful computations, not evidence of optimal costs or a deterministic method of finding the choices. Fibonacci examples are also recorded, but should not be taken as evidence of a new lower-bound separation: Pratt's program is already known to be fast on that family [1, Corollary 6C.7].

Run from the repository root:

```
lake env lean research/RegisterProgram.lean
python3 research/check_register_program.py
python3 -O research/check_register_program.py
python3 research/register_program.py --width 9
```

The program-family bound is now kernel-checked as well as available for independent mathematical scrutiny. The main declarations, all under `ShortGCD.RegisterProgram`, are:

- `maxCost_step`: the fixed-register finite recurrence;
- `Fixed.limsup_profile_le`: the leading coefficient $3/p$;
- `Fixed.eventually_accepts_bound`: the bound for accepted runs, including the final verification;
- `Syntax.eventually_eval_bound`: the same bound for first-order program evaluations;
- `Syntax.arbitrarily_small_program`: the quantified $\varepsilon \log a + B_\varepsilon$ statement with the program fixed before the inputs are chosen.
- `Syntax.Machine.eval_iff_steps`: exact-cost equivalence between evaluation trees and terminating small-step executions;
- `Syntax.Machine.arbitrarily_small_program`: the same quantified bound directly for finite machine transition sequences.

The proof modules are included in the normal `lakebuild` target. The research entry point prints their axiom dependencies, recorded in `register-axioms.txt`; only the standard axioms `propext`, `Classical.choice`, and `Quot.sound` occur. The program-family theorem and its proof are included in Section 6 of the companion manuscript; this note retains the expanded operational discussion and executable checks. The accompanying `model-novelty-audit.txt` records the model comparison and the scope of the primary-literature search. Obtaining one sublogarithmic program remains the larger open task. The separate investigation `single-program-investigation.pdf` tests a fixed program that regenerates selected prefix operands, analyzes its accounting limitation, and identifies a possible structural direction.

References

[1] Yiannis N. Moschovakis. *Abstract Recursion and Intrinsic Complexity*. Lecture Notes in Logic 48, Cambridge University Press / ASL, 2019. https://www.math.ucla.edu/~ynm/lectures/aricplace/c-aric.pdf.

[2] Jason Davies. *The Euclidean algorithm is not optimal among uniform processes*. Companion repository manuscript, October 2026.